

Homework #4

November 30, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

(a)

Let $\beta = (\beta_0, \beta_1, \beta_2, \dots, \beta_K)^T$ be the coefficient vector in $\mathbb{R}^{K+1}$ and $b_0(x) = 1$ be a constant function. Thus we can write f as

$$f(x) = \sum_{k=0}^K \beta_k b_k(x).$$

Let B be the $n \times (K+1)$ matrix

$$B = \begin{pmatrix} b_0(x_1) & b_1(x_1) & b_2(x_1) & \cdots & b_K(x_1) \\ b_0(x_2) & b_1(x_2) & b_2(x_2) & \cdots & b_K(x_2) \\ b_0(x_3) & b_1(x_3) & b_2(x_3) & \cdots & b_K(x_3) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_0(x_n) & b_1(x_n) & b_2(x_n) & \cdots & b_K(x_n) \end{pmatrix}$$

and $y = (y_1, y_2, \dots, y_n)^T$ be the response vector in $\mathbb{R}^n$. Then

$$\|B\beta - y\|_2^2 = \sum_{i=1}^n \left(\sum_{k=0}^K \beta_k b_k(x_i) - y_i \right)^2 = \sum_{i=1}^n (f(x_i) - y_i)^2.$$

So $\|B\beta - y\|_2^2$ is the desired matrix/vector form of the first term of the objective function. In pure matrix form, this is written as

$$\begin{aligned} \|B\beta - y\|_2^2 &= (B\beta - y)^T (B\beta - y) \\ &= \beta^T B^T B \beta - 2y^T B \beta + y^T y. \end{aligned}$$

(b)

Let A be the $(K + 1) \times (K + 1)$ (symmetric) matrix

$$A = \begin{pmatrix} \int b_0''(x)b_0''(x)dx & \int b_0''(x)b_1''(x)dx & \int b_0''(x)b_2''(x)dx & \cdots & \int b_0''(x)b_K''(x)dx \\ \int b_1''(x)b_0''(x)dx & \int b_1''(x)b_1''(x)dx & \int b_1''(x)b_2''(x)dx & \cdots & \int b_1''(x)b_K''(x)dx \\ \int b_2''(x)b_0''(x)dx & \int b_2''(x)b_1''(x)dx & \int b_2''(x)b_2''(x)dx & \cdots & \int b_2''(x)b_K''(x)dx \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \int b_K''(x)b_0''(x)dx & \int b_K''(x)b_1''(x)dx & \int b_K''(x)b_2''(x)dx & \cdots & \int b_K''(x)b_K''(x)dx \end{pmatrix}.$$

Then we can write

$$\begin{aligned} \int (f''(x))^2 dx &= \int \left(\sum_{k=0}^K \beta_k b_k''(x) \right)^2 dx \\ &= \int \left(\sum_{i=0}^K \sum_{j=0}^K \beta_i \beta_j b_i''(x) b_j''(x) \right) dx \\ &= \sum_{i=0}^K \sum_{j=0}^K \beta_i \beta_j \int b_i''(x) b_j''(x) dx \\ &= \beta^T A \beta. \end{aligned}$$

(c)

Combining part (a) and (b), the objective function is

$$\beta^T B^T B \beta - 2y^T B \beta + y^T y + \lambda \beta^T A \beta = \beta^T (B^T B + \lambda A) \beta - 2y^T B \beta + y^T y.$$

At its minimum the gradient must be zero, so

$$-2B^T y + ((B^T B + \lambda A) + (B^T B + \lambda A)^T) \beta = 2(-B^T y + (B^T B + \lambda A) \beta) = 0,$$

which solves to $\beta = (B^T B + \lambda A)^{-1} B^T y$.

2.

(a), (b)

Below we load the data and fit a smoothing spline to it. Although the best λ given by the cross validation is around 0.48939, the resulting prediction function does not look smooth enough. A larger smoothing parameter, say $\lambda = 10$, would result in a more believable yield curve.

```
[3]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```

from pygam import LinearGAM

rates = pd.read_csv('forward_rates.csv')
x = rates['time'].values
y = rates['rate'].values

x_grid = np.linspace(0, 30, num=1000)
gam = LinearGAM().gridsearch(x.reshape((-1, 1)), y, lam=np.logspace(-5, 3, 30),
    ↪keep_best=True)
gam10 = LinearGAM(lam=10).fit(x.reshape((-1, 1)), y)
print('Optimal lambda: ', gam.lam[0][0])

fig, axs = plt.subplots(1, 2, figsize=(12, 4))

sns.scatterplot(x, y, ax=axs[0])
sns.lineplot(x_grid, gam.predict(x_grid), ax=axs[0], color='r')
axs[0].set(xlabel='Term (Year)', ylabel='Forward Rate', title='$\lambda=0.
    ↪48939$ (The Optimal $\lambda$ by Cross Validation)')

sns.scatterplot(x, y, ax=axs[1])
sns.lineplot(x_grid, gam10.predict(x_grid), ax=axs[1], color='r')
axs[1].set(xlabel='Term (Year)', ylabel='Forward Rate', title='$\lambda = 10$')

plt.suptitle('Forward Rates and Smoothing Splines', fontsize=16)
plt.subplots_adjust(top=0.80)
plt.show()

```

100% (30 of 30) |#####| Elapsed Time: 0:00:00 Time: 0:00:00
/srv/conda/envs/notebook/lib/python3.7/site-packages/seaborn/_decorators.py:43:
FutureWarning: Pass the following variables as keyword args: x, y. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or
misinterpretation.

FutureWarning
/srv/conda/envs/notebook/lib/python3.7/site-packages/seaborn/_decorators.py:43:
FutureWarning: Pass the following variables as keyword args: x, y. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or
misinterpretation.

FutureWarning
Optimal lambda: 0.4893900918477489

/srv/conda/envs/notebook/lib/python3.7/site-packages/seaborn/_decorators.py:43:
FutureWarning: Pass the following variables as keyword args: x, y. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or

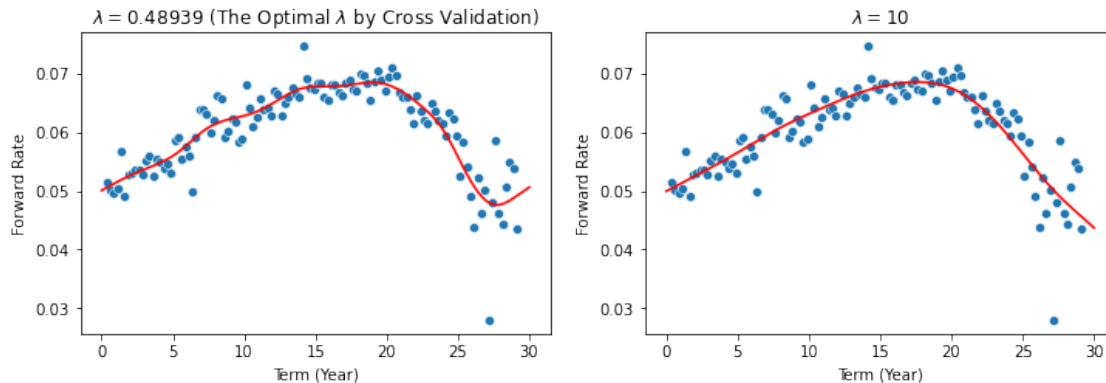
misinterpretation.

FutureWarning

/srv/conda/envs/notebook/lib/python3.7/site-packages/seaborn/_decorators.py:43: FutureWarning: Pass the following variables as keyword args: x, y. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

FutureWarning

Forward Rates and Smoothing Splines



(c)

A cubic spline is a piecewise cubic polynomial so its definite integral is known in closed form. Although it takes time to locate which “piece” the limits of integration are in ($O(n)$ or $O(\log n)$, depending on whether the knots are sorted, where n is the number of knots), it is computationally very cheap compared to numerical integration like Simpson’s rule.

3.

(a)

First we confirm there is no missing data in the data sets.

```
[1]: import pandas as pd

train = pd.read_csv('bike_train.csv')
test = pd.read_csv('bike_test.csv')

train.isna().any().any(), test.isna().any().any()
```

```
[1]: (False, False)
```

An OLS is fitted and used to predict the rentals. The prediction is then multiplied by 1.6 to account for the ridership growth. The MSE is 1295159.395 as shown below.

```
[2]: from sklearn.metrics import mean_squared_error
import statsmodels.api as sm

X_train = pd.read_csv('bike_train.csv')
y_train = X_train.pop('rentals')

ols = sm.OLS(exog=sm.add_constant(X_train), endog=y_train).fit()

X_test = pd.read_csv('bike_test.csv')
y_test = X_test.pop('rentals')

yhat_ols = ols.predict(exog=sm.add_constant(X_test))*1.6
mean_squared_error(yhat_ols, y_test)
```

```
[2]: 1295159.3945082375
```

(b)

Below we fit additive models through a grid search to find the best smoothing parameter combination. We then multiply the prediction by 1.6 and use the results to compute the MSE. The optimal MSE turns out to be 974017.893.

```
[5]: from pygam import LinearGAM, s, f
import numpy as np

gam = LinearGAM(s(0)+f(1)+f(2)+f(3)+s(4)+s(5)+s(6), fit_intercept=True).
    ↳gridsearch(X_train.values, y_train.values, lam=[np.logspace(-1, 2, 3)]*7)
```

```
100% (2187 of 2187) |#####| Elapsed Time: 0:22:30 Time: 0:22:30
```

```
[10]: gam.summary()
```

```
LinearGAM
=====
Distribution:                      NormalDist Effective DoF:
37.6924
Link Function:                     IdentityLink Log Likelihood:
-4811.8819
Number of Samples:                 365 AIC:
9701.1487
                                     AICc:
9710.5908
                                     GCV:
```

259785.4598

Scale:

211990.7438

Pseudo R-Squared:

0.8997

Feature Function		Lambda	Rank	EDoF
P > x	Sig. Code			
s(0)		[0.1]	20	16.0
1.11e-16	***			
f(1)		[100.]	7	2.6
2.90e-01				
f(2)		[100.]	2	0.2
1.89e-01				
f(3)		[0.1]	3	2.0
8.01e-06	***			
s(4)		[3.1623]	20	7.6
1.11e-16	***			
s(5)		[3.1623]	20	6.9
7.63e-11	***			
s(6)		[100.]	20	2.5
3.88e-07	***			
intercept			1	0.0
1.11e-16	***			

Significance codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

WARNING: Fitting splines and a linear function to a feature introduces a model identifiability problem

which can cause p-values to appear significant when they are not.

WARNING: p-values calculated in this manner behave correctly for un-penalized models or models with

known smoothing parameters, but when smoothing parameters have been estimated, the p-values

are typically lower than they should be, meaning that the tests reject the null too readily.

/srv/conda/envs/notebook/lib/python3.7/site-packages/ipykernel_launcher.py:1:

UserWarning: KNOWN BUG: p-values computed in this summary are likely much smaller than they should be.

Please do not make inferences based on these values!

Collaborate on a solution, and stay up to date at:
github.com/dswah/pyGAM/issues/163

"""Entry point for launching an IPython kernel.

```
[5]: from pygam import LinearGAM, s, f

gam = LinearGAM(s(0)+f(1)+f(2)+f(3)+s(4)+s(5)+s(6), fit_intercept=True, lam=[0.
↪1, 100, 100, 0.1, 3.1623, 3.1623, 100]).fit(X_train.values, y_train.values)

yhat_gam = gam.predict(X_test)*1.6
mean_squared_error(yhat_gam, y_test)
```

```
[5]: 974017.8932338969
```

(c)

Below are the function plots:

```
[10]: import matplotlib.pyplot as plt

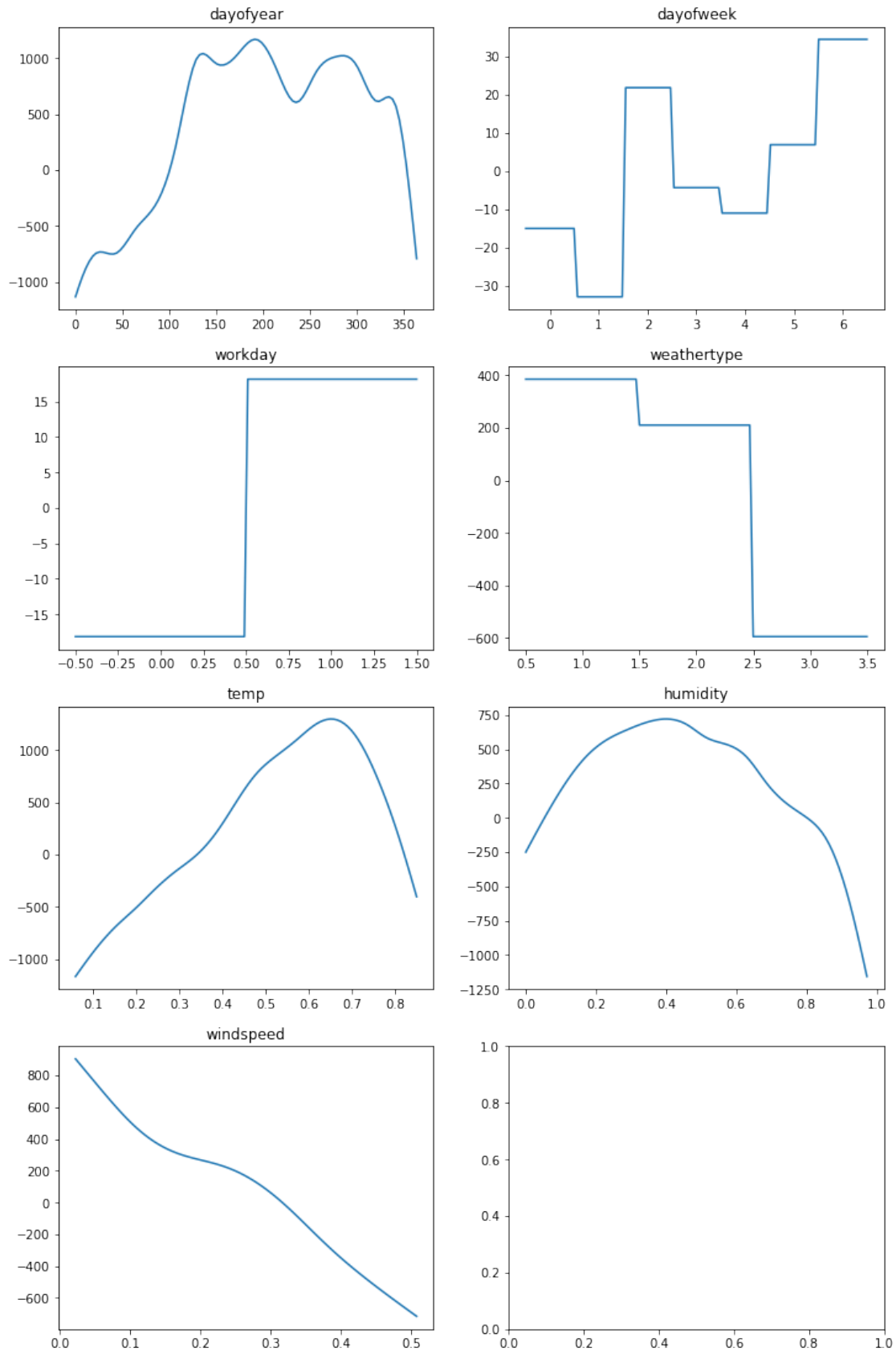
fig, axs = plt.subplots(4, 2, figsize=(12, 18))

for i, ax in enumerate(axs.flatten()[:7]):
    Xfake = gam.generate_X_grid(i)
    if i != 3:
        Xfake[:, 3] += 1

    pdep, confi = gam.partial_dependence(term=i, X=Xfake, width=.95)
    ax.plot(Xfake[:, i], pdep)
    ax.set_title(X_train.columns[i])

plt.suptitle('Fitted Terms of the Additive Model', fontsize=16)
plt.subplots_adjust(top=0.94)
plt.show()
```

Fitted Terms of the Additive Model



windspeed and workday

According to the fitted model, the more windy it is, the less rentals there are, which makes sense. There are more rentals on workdays than weekend/holidays. One possible explanation is most of the rentals are for commute. Those are the only two features that have close-to-linear relationships to the response that could be captured by OLS.

temp and humidity

The dependence on temperature and humidity also makes sense. If it is too cold or too hot, there will be less rentals; if it is too dry or too humid, there will be less rentals. Those are nonlinear behavior OLS will not be able to capture.

weathertype

The resulting fit is as expected: the better the weather is, the more rentals there are.

Although from the data description there are 4 weather types, the last one seems to be extremely rare and is not observed in either training or test data.

```
[37]: print('Weather types observed in 2011: ', X_train['weathertype'].unique())  
      print('Weather types observed in 2012: ', X_test['weathertype'].unique())
```

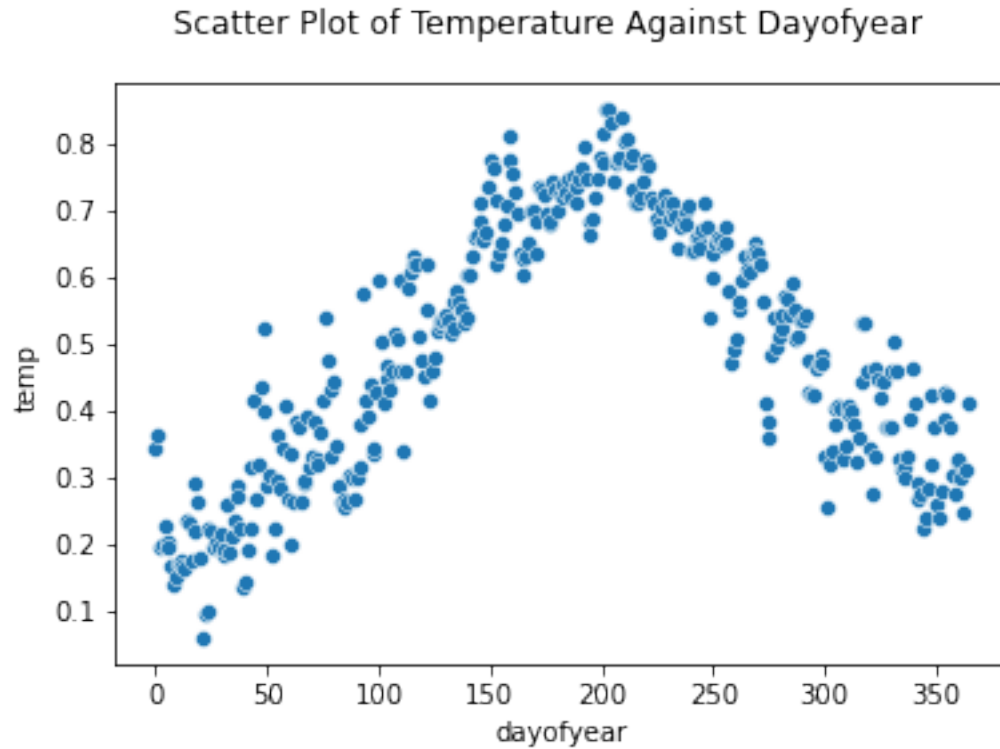
```
Weather types observed in 2011:  [2 1 3]
```

```
Weather types observed in 2012:  [1 2 3]
```

dayofyear

It looks like the response depends on dayofyear through temperature. Below is the scatter plot of temperature against dayofyear. There are less rentals in the winter when the temperature is low. There are more rentals on other seasons but in the summer (around the 200th day) the rentals drop slightly at the temperature peaks.

```
[11]: sns.scatterplot(x='dayofyear', y='temp', data=X_train)  
      plt.suptitle('Scatter Plot of Temperature Against Dayofyear')  
      plt.show()
```



dayofweek

The description of the data does not specify but, from the fit, most likely the counting starts from Monday (meaning 0 is Monday). This is also the convention used in the python `datetime` module. Not surprisingly there are most rentals on Sundays. But from the training data the second best day of week in terms of bike rentals is Wednesday. This pattern disappears in the test data. Below are box plots of rentals for different days of week in 2011 and 2012.

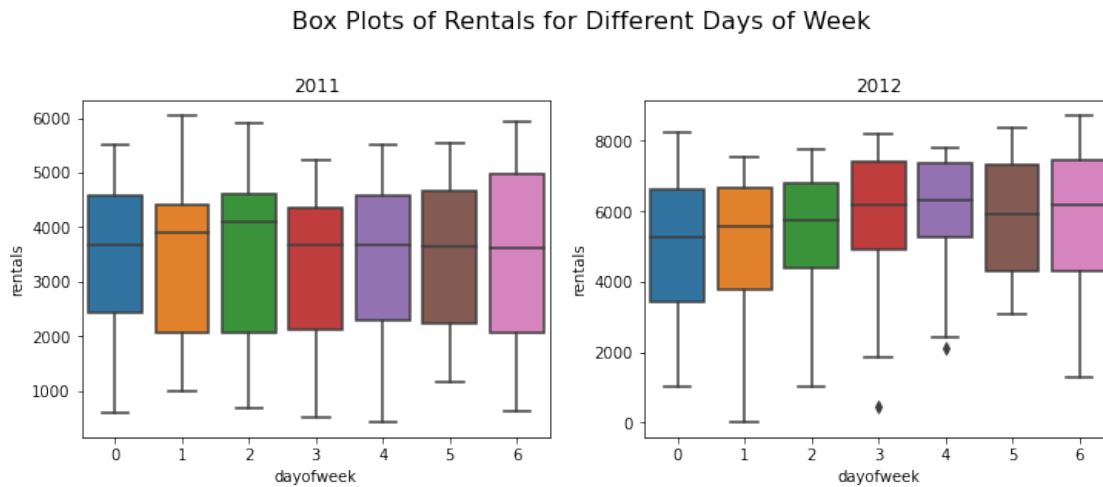
```
[28]: import pandas as pd
import seaborn as sns

fig, axs = plt.subplots(1, 2, figsize=(12, 4.5))

sns.boxplot(x='dayofweek', y='rentals', data=pd.read_csv('bike_train.csv'),
            ↪ax=axs[0])
sns.boxplot(x='dayofweek', y='rentals', data=pd.read_csv('bike_test.csv'),
            ↪ax=axs[1])

fig.suptitle('Box Plots of Rentals for Different Days of Week', fontsize=16)
axs[0].set(title='2011')
axs[1].set(title='2012')
plt.subplots_adjust(top=0.80)
```

```
plt.show()
```



(d), (e)

The below table gives the average loss computed by the new asymmetric loss function.

With the quantile adjustment, both OLS and GAM results improve. The GAM is better than OLS with and without the adjustment. The best estimator is GAM with adjustment.

```
[46]: from pandas import DataFrame

def read_loss(true, guess):
    return (np.mean(np.where(true>guess, 5*(true-guess), guess-true)))

quantile_ols = np.percentile(y_test - yhat_ols, 100*5/6)
quantile_gam = np.percentile(y_test - yhat_gam, 100*5/6)

DataFrame({ 'OLS': [read_loss(y_test, yhat_ols), read_loss(y_test, yhat_ols +
↪quantile_ols)],
            'GAM': [read_loss(y_test, yhat_gam), read_loss(y_test, yhat_gam +
↪quantile_gam)] },

            index=['without quantile adjustment', 'with quantile adjustment'])
```

	OLS	GAM
without quantile adjustment	2412.830285	1988.925841
with quantile adjustment	1429.453400	1348.070859

4.

(a)

The expected prediction error is

$$\begin{aligned}\text{EPE} &= \mathbb{E} \left[\mathcal{L}(Y, \hat{f}(X)) \right] \\ &= \mathbb{E} \left[\mathbb{E} \left[\mathcal{L}(Y, \hat{f}(X)) | X \right] \right] \\ &= \mathbb{E} \left[\mathbb{P}(Y = 1 | X) \mathcal{L}(1, \hat{f}(X)) + \mathbb{P}(Y = 0 | X) \mathcal{L}(0, \hat{f}(X)) \right].\end{aligned}$$

For any fixed $X = x$, we minimize the inside of the expectation over $\hat{f}$:

$$\begin{aligned}\hat{f}(x) &= \underset{g \in \{0,1\}}{\operatorname{argmin}} [\mathbb{P}(Y = 1 | X = x) \mathcal{L}(1, g) + \mathbb{P}(Y = 0 | X = x) \mathcal{L}(0, g)] \\ &= \begin{cases} 1 & \text{if } \mathbb{P}(Y = 1 | X = x) \mathcal{L}(1, 0) + \mathbb{P}(Y = 0 | X = x) \mathcal{L}(0, 0) > \mathbb{P}(Y = 1 | X = x) \mathcal{L}(1, 1) + \mathbb{P}(Y = 0 | X = x) \mathcal{L}(0, 1) \\ 0 & \text{otherwise} \end{cases} \\ &= \begin{cases} 1 & \text{if } \mathbb{P}(Y = 1 | X = x) \mathcal{L}(1, 0) > \mathbb{P}(Y = 0 | X = x) \mathcal{L}(0, 1) \\ 0 & \text{otherwise} \end{cases} \\ &= \begin{cases} 1 & \text{if } \mathbb{P}(Y = 1 | X = x) L_{10} > \mathbb{P}(Y = 0 | X = x) L_{01} \\ 0 & \text{otherwise} \end{cases}.\end{aligned}$$

Assuming $L_{10} > 0, L_{01} > 0$, the condition $\mathbb{P}(Y = 1 | X = x) L_{10} > \mathbb{P}(Y = 0 | X = x) L_{01}$ can be further simplified as follows:

$$\begin{aligned}\mathbb{P}(Y = 1 | X = x) L_{10} &> \mathbb{P}(Y = 0 | X = x) L_{01} \\ \iff \mathbb{P}(Y = 1 | X = x) L_{10} &> (1 - \mathbb{P}(Y = 1 | X = x)) L_{01} \\ \iff \mathbb{P}(Y = 1 | X = x) (L_{10} + L_{01}) &> L_{01} \\ \iff \mathbb{P}(Y = 1 | X = x) &> \frac{L_{01}}{L_{10} + L_{01}}.\end{aligned}$$

Thus the optimal optimization rule is

$$\hat{f}(x) = \begin{cases} 1 & \text{if } \mathbb{P}(Y = 1 | X = x) > \frac{L_{01}}{L_{10} + L_{01}} \\ 0 & \text{otherwise} \end{cases}.$$

(b)

The classification rule given 0-1 loss is a special case of the above optimal rule. Indeed, given the 0-1 loss we have $L_{10} = L_{01}$ and therefore the threshold is $\frac{L_{01}}{L_{10} + L_{01}} = 0.5$.

If $L_{10} > L_{01}$, the threshold $\frac{L_{01}}{L_{10} + L_{01}}$ will be less than 0.5, in which case even if the conditional probability $\mathbb{P}(Y = 1 | X = x)$ is less than 0.5, as long as it is still greater than the threshold, the classifier will guess $Y = 1$. Thus it has a tendency to choose $Y = 1$ over $Y = 0$, which makes sense when false negatives are worse than false positives.